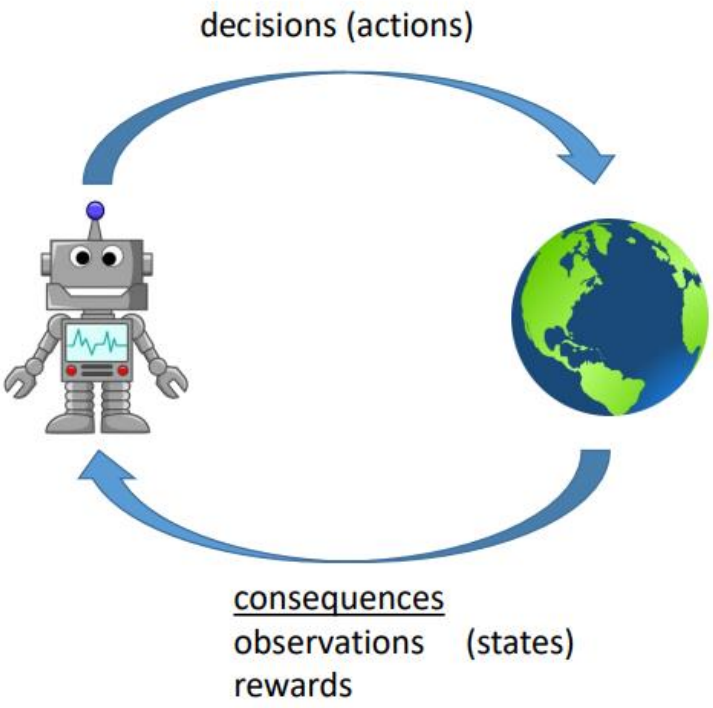
Reinforcement Learning – Other Topics

Francisco Giral

A Recap of what RL is..



Actions: muscle contractions
Observations: sight, smell
Rewards: food

Actions: motor current or torque
Observations: camera images
Rewards: task success measure (e.g., running speed)

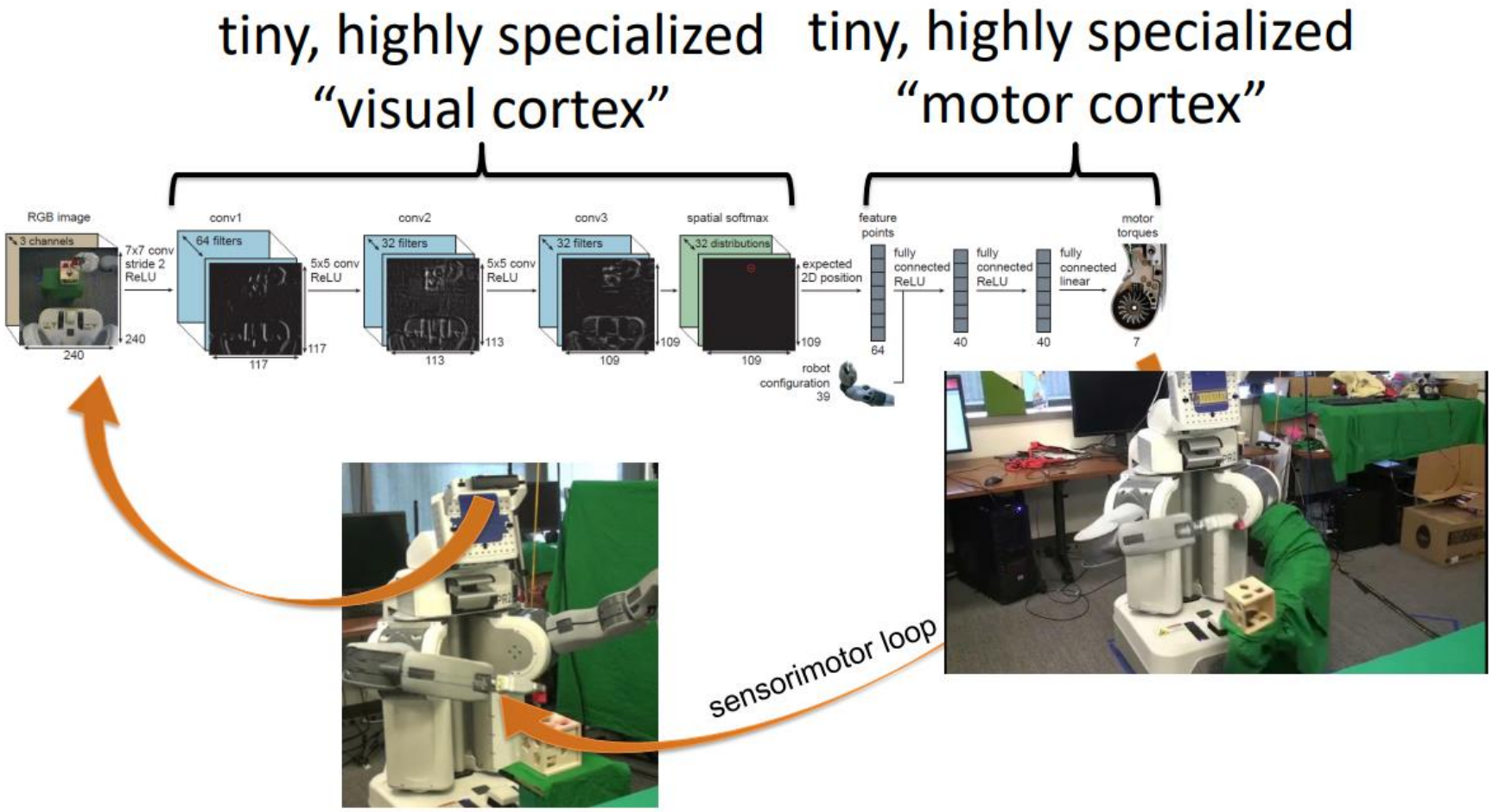
Actions: what to purchase
Observations: inventory levels
Rewards: profit

A Recap of what RL is..

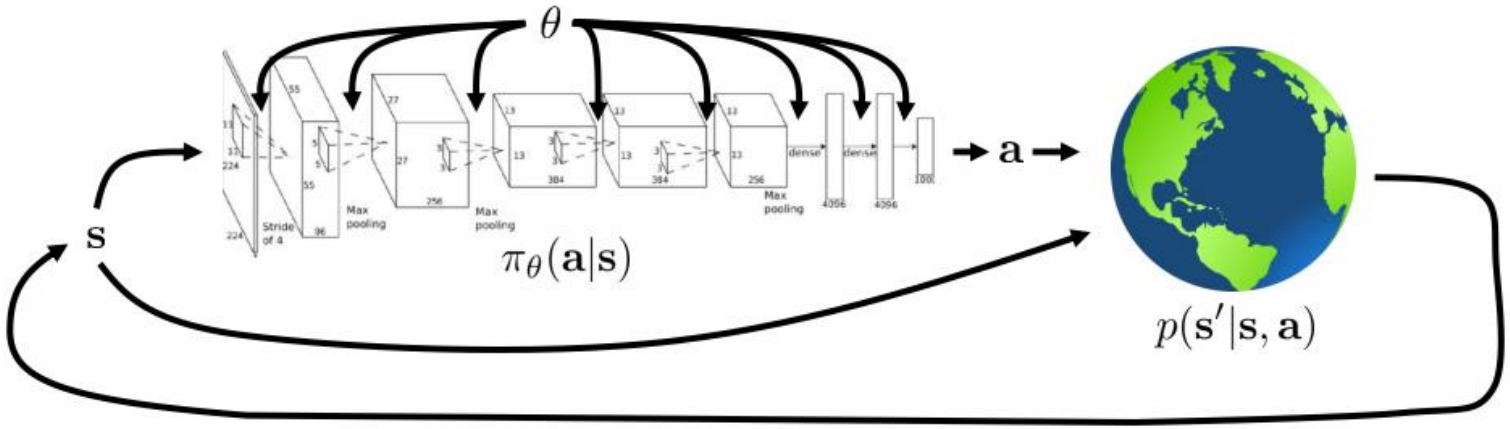
Why should we care about deep reinforcement learning?

RL seems the way to intelligent machines...

A Recap of what RL is..



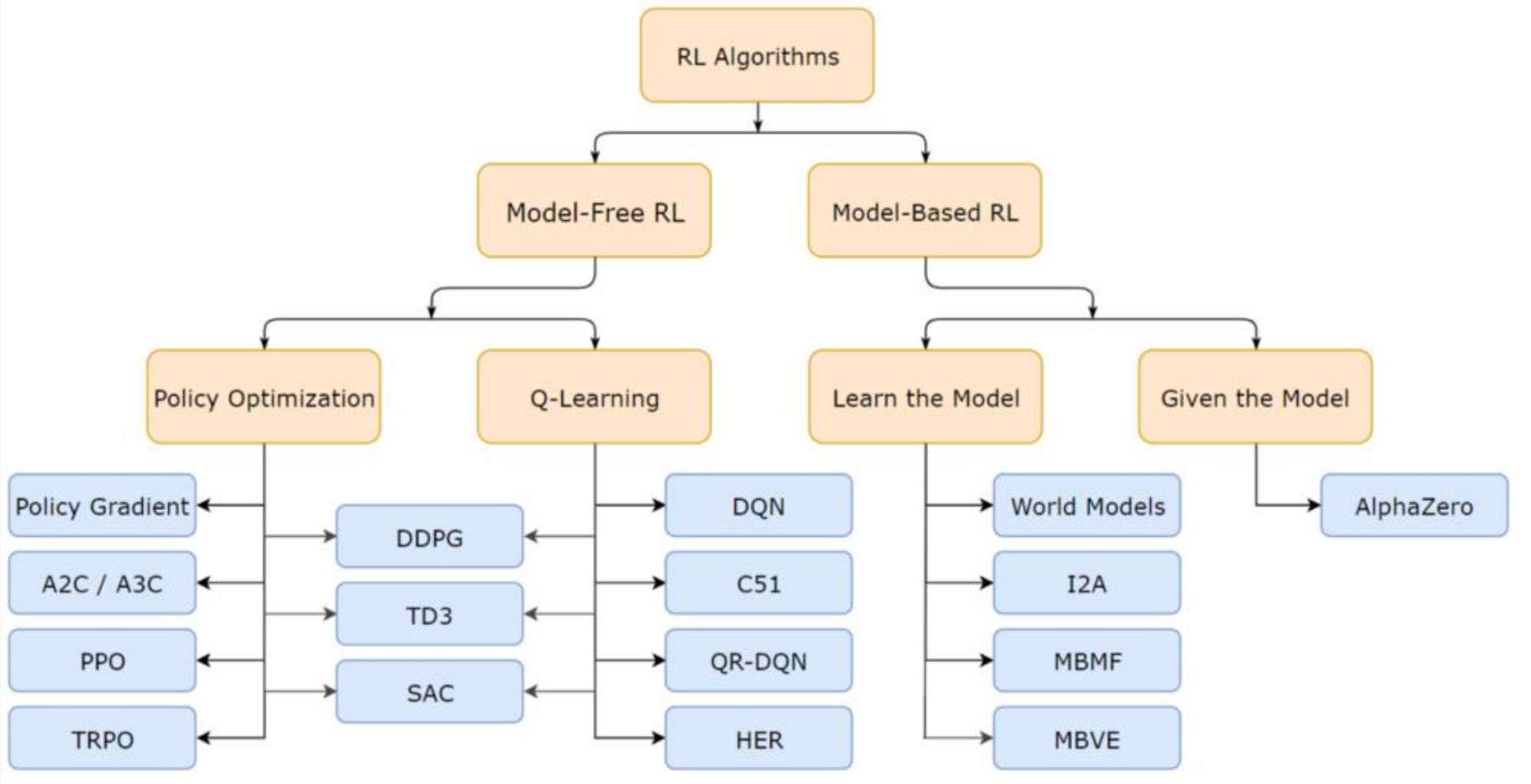
A Recap of what RL is..



$$\underbrace{p_\theta(\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)}_{\pi_\theta(\tau)} = p(\mathbf{s}_1) \prod_{t=1}^{T} \pi_\theta(\mathbf{a}_t|\mathbf{s}_t) p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$$

$$\theta^\star = \arg\max_\theta E_{\tau \sim p_\theta(\tau)} \left[\sum_t r(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Algorithms Taxonomy

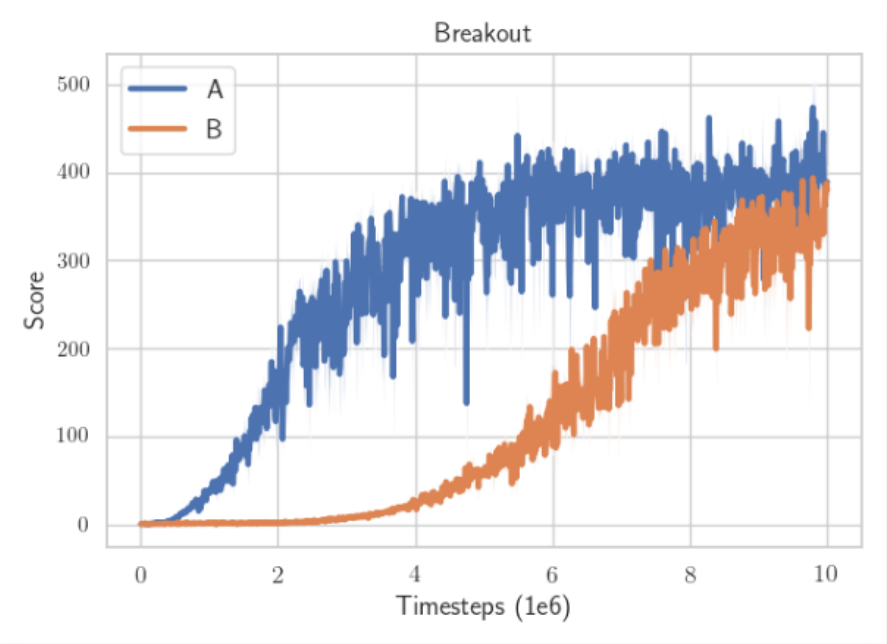


Topics..

- **Tips and Tricks for Practical RL**
- **Advanced Topics:**
  - Model-based Reinforcement Learning
  - Imitation Learning
  - RL in Language Models
  - Multi-Agent RL

Practical Implications of RL

RL is hard



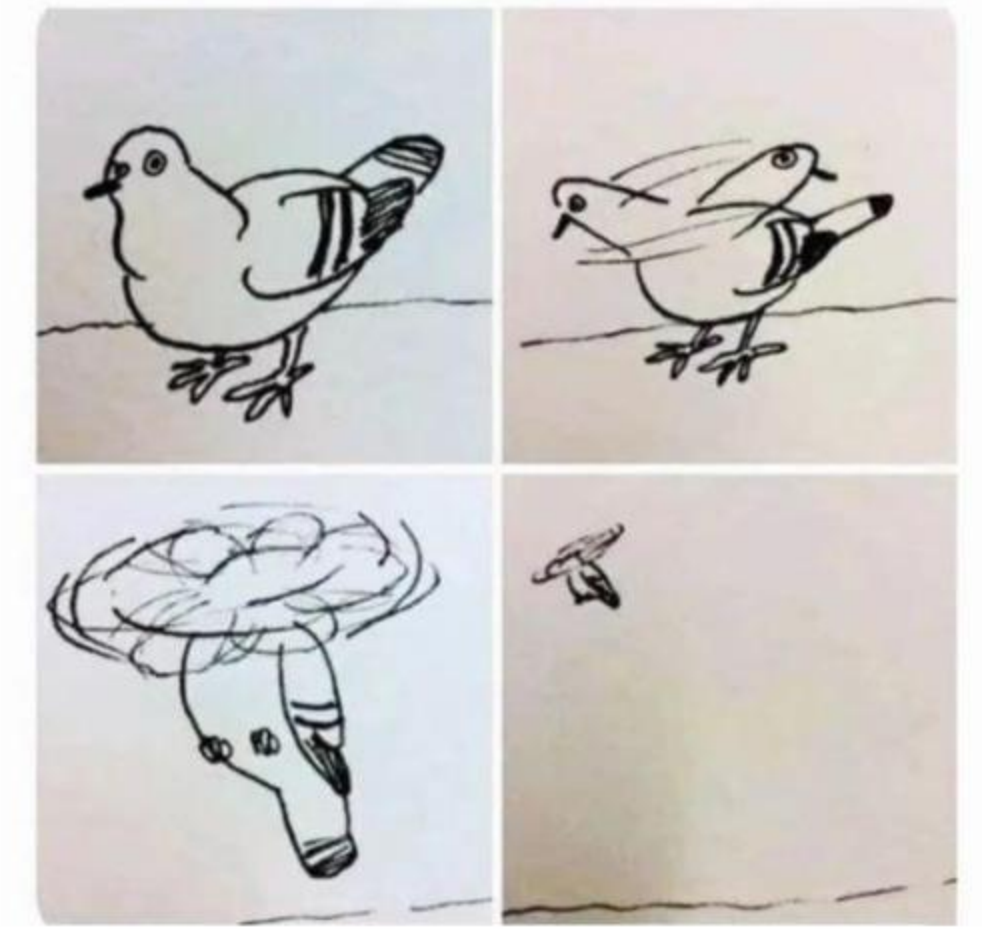
The only difference: the epsilon ϵ value to avoid division by zero in the optimizer
(one is ϵ = 1e-7 the other ϵ = 1e-5)

Which algorithm is better?

RL is hard

- data collection by the agent itself
- sensitivity to the random seed / hyperparameters
- sample inefficient
- reward function design

When your RL agent learns to glitch the environment



The best library to begin with RL

STABLE-BASELINES3

Reliable RL Implementations

```python
import gym
from stable_baselines3 import SAC
# Train an agent using Soft Actor-Critic on Pendulum-v0
env = gym.make("Pendulum-v0")
model = SAC("MlpPolicy", env).learn(total_timesteps=20000)
# Save the model
model.save("sac_pendulum")
# Load the trained model
model = SAC.load("sac_pendulum")
# Start a new episode
obs = env.reset()
# What action to take in state `obs`?
action, _ = model.predict(obs, deterministic=True)
```

https://github.com/DLR-RM/stable-baselines3

Other useful libraries

URL	Language	Comments
Stoix	Jax	Mini-library with many methods (including MBRL)
PureJaxRL	Jax	Single files with DQN; PPO, DPO
JaxRL	Jax	Single files with AWAC, DDPG, SAC, SAC+REDQ
Stable Baselines Jax	Jax	Library with DQN, CrossQ, TQC; PPO, DDPG, TD3, SAC
Jax Baselines	Jax	Library with many methods
Rejax	Jax	Library with DDQN, PPO, (discrete) SAC, DDPG
Dopamine	Jax/TF	Library with many methods
Rlax	Jax	Library of RL utility functions (used by Acme)
Acme	Jax/TF	Library with many methods (uses rlax)
CleanRL	PyTorch	Single files with many methods
Stable Baselines 3	PyTorch	Library with DQN; A2C, PPO, DDPG, TD3, SAC, HER
TianShou	PyTorch	Library with many methods (including offline RL)

Table 1.3: Some open source RL software.

Tips for environment definition

Environment ---> All that changes if the agent takes an action

Examples:

- **Flight Control:** Aircraft's dynamics and surroundings
- **Flow Control:** State of the fluid flow field
- **Robotics:** All states of the robot and its surroundings

Tips for environment definition

```python
import gymnasium as gym
import numpy as np
from gymnasium import spaces


class CustomEnv(gym.Env):
    """Custom Environment that follows gym interface."""

    metadata = {"render_modes": ["human"], "render_fps": 30}

    def __init__(self, arg1, arg2, ...):
        super().__init__()
        # Define action and observation space
        # They must be gym.spaces objects
        # Example when using discrete actions:
        self.action_space = spaces.Discrete(N_DISCRETE_ACTIONS)
        # Example for using image as input (channel-first; channel-last also works):
        self.observation_space = spaces.Box(low=0, high=255,
                                            shape=(N_CHANNELS, HEIGHT, WIDTH), dtype=np.uint8)

    def step(self, action):
        ...
        return observation, reward, terminated, truncated, info

    def reset(self, seed=None, options=None):
        ...
        return observation, info

    def render(self):
        ...

    def close(self):
        ...
```

Action Space

Discrete --> UP, DOWN, RIGHT, LEFT....

Multi-Discrete --> for example if 4: [DOWN, RIGHT, RIGHT, LEFT]

Continuous --> Gaussian Distribution per action [-1, 1]

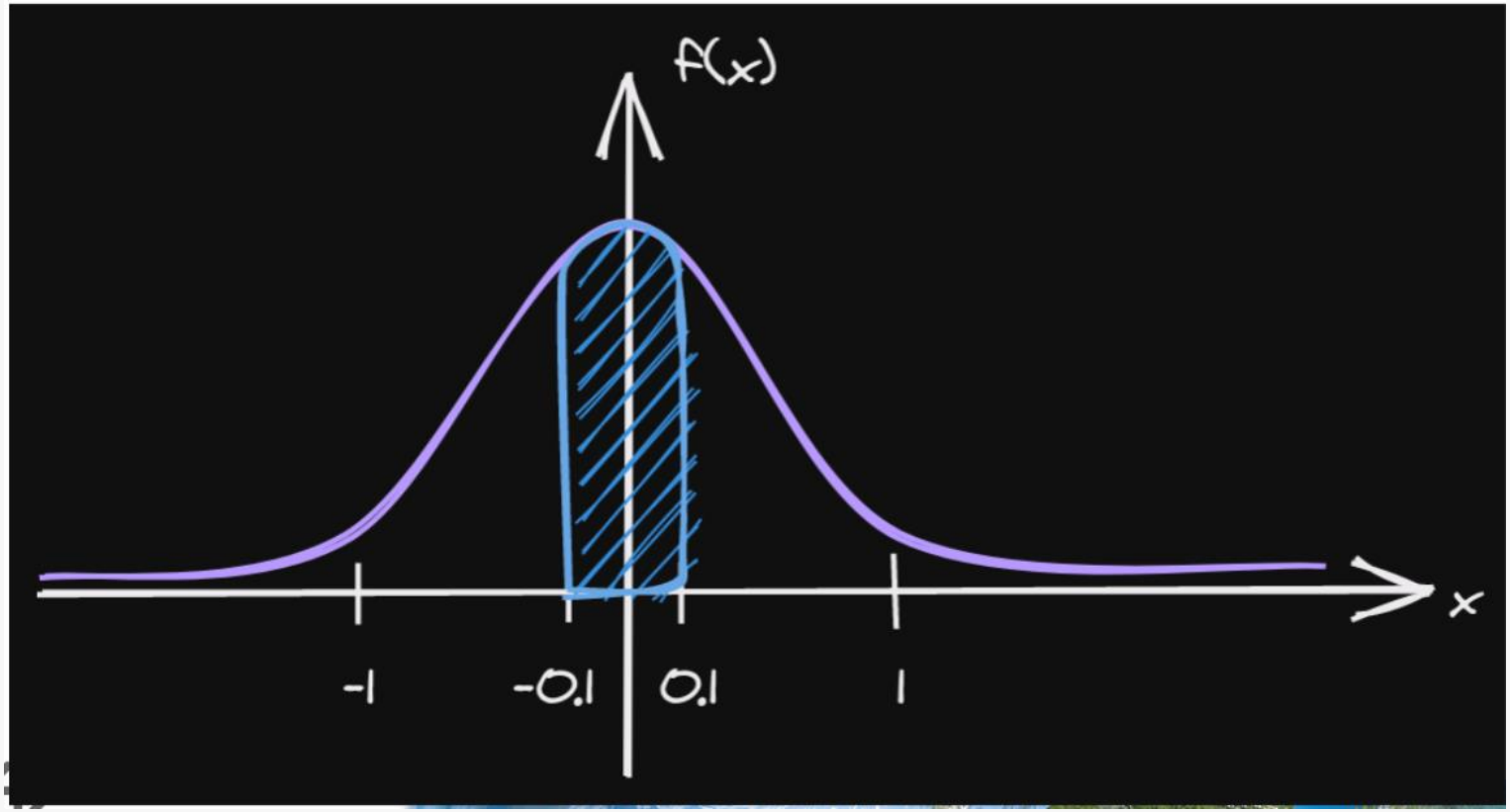
Action Space

CONTINUOUS ACTION SPACE: NORMALIZE?
NORMALIZE!

```python
from gym import spaces

# Unnormalized action spaces only work with algorithms
# that don't directly rely on a Gaussian distribution to define the policy
# (e.g. DDPG or SAC, where their output is rescaled to fit the action space limits)

# LIMITS TOO BIG: in that case, the sampled actions will only have values
# around zero, far away from the limits of the space
action_space = spaces.Box(low=-1000, high=1000, shape=(n_actions,), dtype="float32")

# LIMITS TOO SMALL: in that case, the sampled actions will almost
# always saturate (be greater than the limits)
action_space = spaces.Box(low=-0.02, high=0.02, shape=(n_actions,), dtype="float32")

# BEST PRACTICE: action space is normalized, symmetric
# and has an interval range of two,
# which is usually the same magnitude as the initial standard deviation
# of the Gaussian used to sample actions (unit initial std in SB3)
action_space = spaces.Box(low=-1, high=1, shape=(n_actions,), dtype="float32")
```

Action Space

CONTINUOUS ACTION SPACE: NORMALIZE?
NORMALIZE!



Observation Space

- enough information to solve the task

- do not break Markov assumption

"The future is conditionally independent of the past, given the present"

- Normalize!
  - Standard
  - Max/min

Observation Space

In many real-world scenarios:

Markov Decision Process (MDP)

Partially Observable MDP (POMDP)

Observation Space

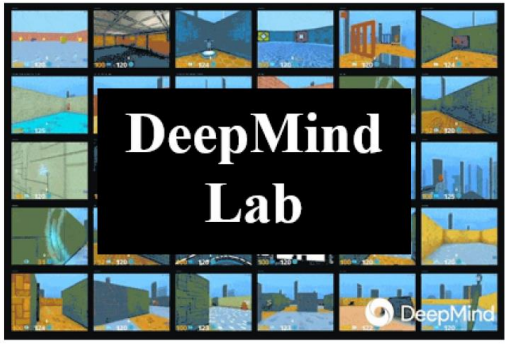
In many real-world scenarios:

Markov Decision Process (MDP)

Partially Observable MDP (POMDP)

We cannot observe the full state

Observation Space



Observation Space

If we get noisy observations, we also get a POMDP!!

Observation Space

What can we do to handle POMDPs?

We give "memory" to the model

Option 1: Stack a history of past observations

Option 2: Use a recurrent policy: LSTM, GRU

Example: Recurrent-PPO

Observation Space

The *Policy* can be, in principle, any kind of model (DNN, CNN, RNN, etc)

However...

We tend to use small nets to avoid gradient variance that comes from the online nature of RL

Reward Signal Design

- The reward function should tell the agent **what to do** but **not how to do it**

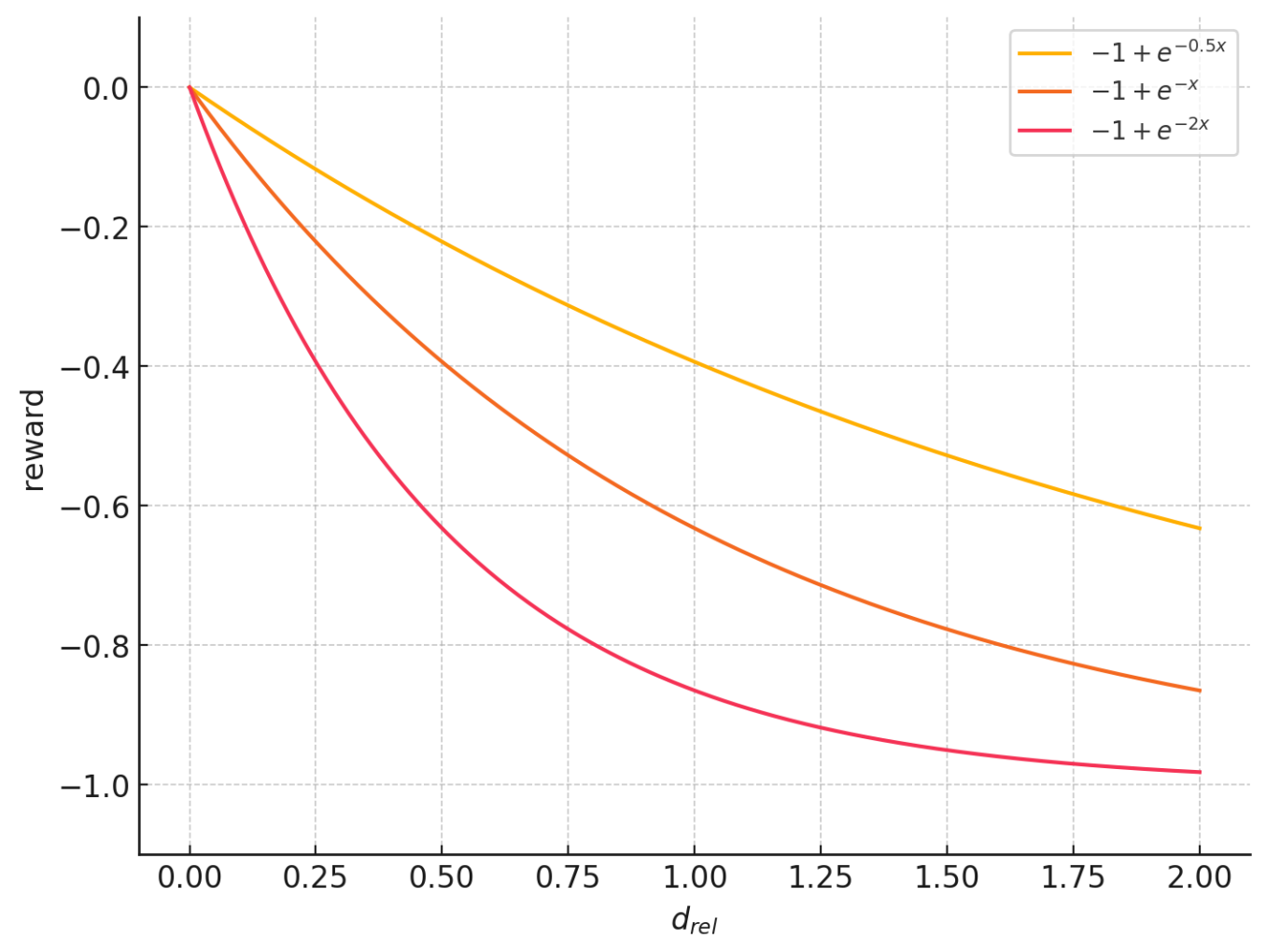
- Rewards can be:
  - **Sparse:** Only when reaching the goal
  - **Shaped:** At each time step towards the goal (reward shaping)

- For practical implementations, the reward is **defined by time step**, not by episode

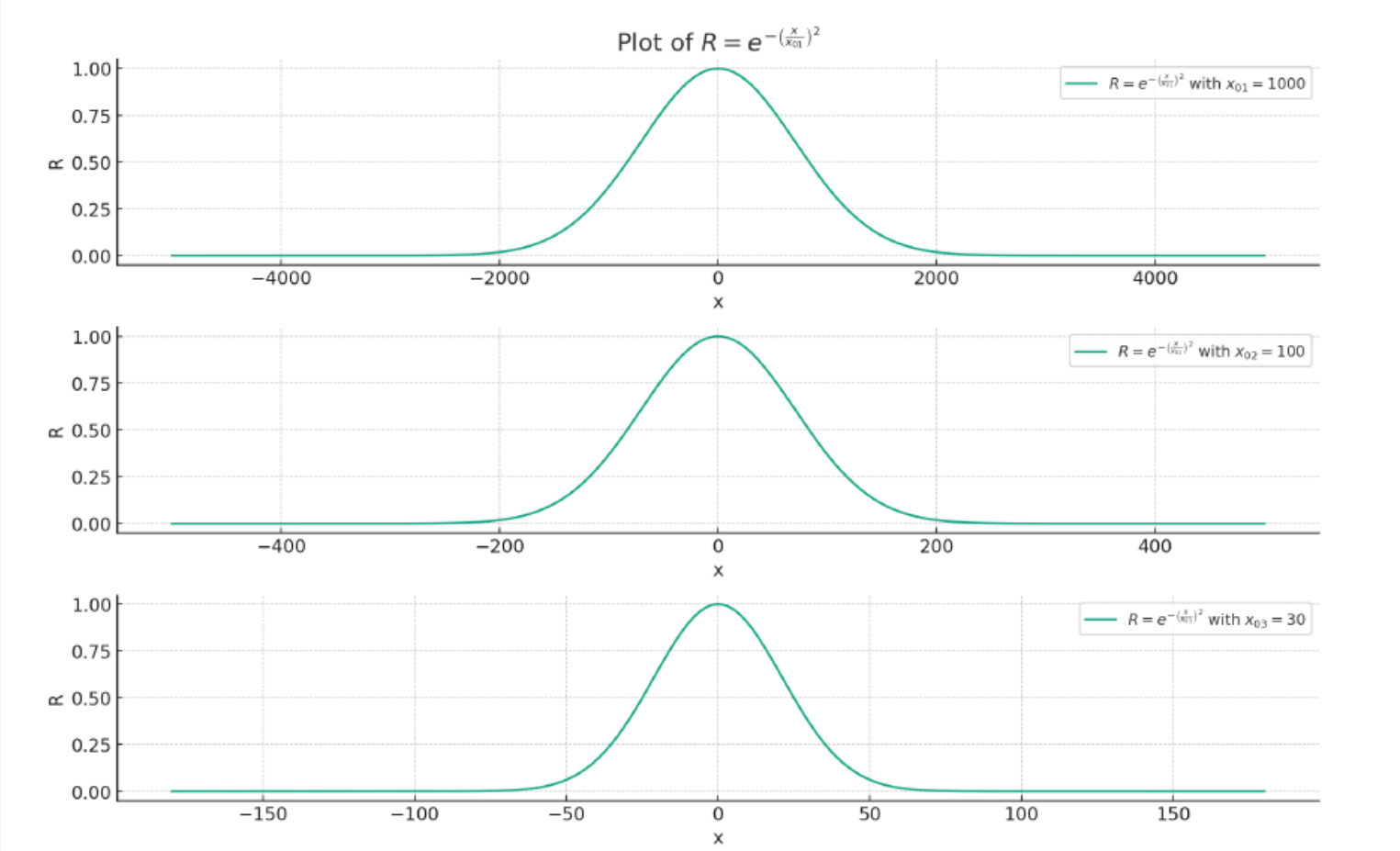
Reward Signal Design

- Choosing the mathematical function:
  - Linear
  - Quadratic
  - Exponential
  - Logarithmic
- You can reward or penalize an action

Reward Signal Design



Reward Signal Design



Termination Conditions

- **Termination:** We end the episode because either the agent reached the goal or it hit a killing condition:
  - It is going too out of the limits
  - It hits a wall
  - It is diverging, etc

  Typically, we penalize when one of these conditions is triggered

  Or we reward if the agent reaches the goal

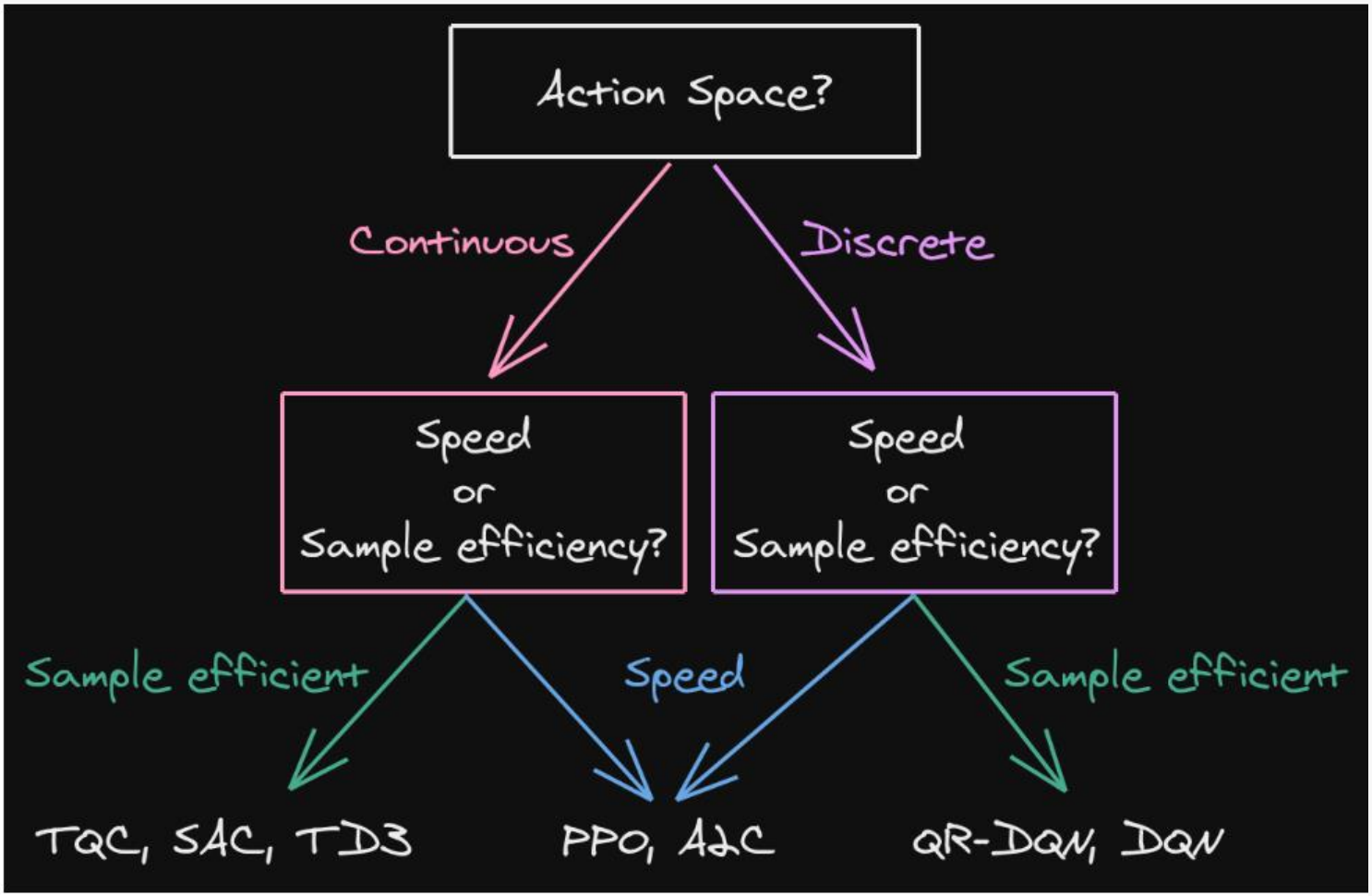
Termination Conditions

- **Truncation:** We end the episode after a certain number of steps (time limit)

- We do this to avoid infinite episodes

- It should be carefully managed if the time is part of the observations

Choosing the right algorithm



Choosing the right algorithm

- Within Model-Free RL we have:
  - On-policy: Faster sampling, less sample efficient (PPO, TRPO)
  - Off-policy: Slower sampling, more sample efficient (SAC, DQN, etc)

Choosing the right algorithm

It depends on your problem:

- ¿Is your simulator fast? --> On-policy
- ¿Do you have little data capacity? --> Off-policy

Choosing the right algorithm

Available algos in SB3

Name	`Box`	`Discrete`	`MultiDiscrete`	`MultiBinary`	Multi Processing
ARS [1]	✔	✔	✘	✘	✔
A2C	✔	✔	✔	✔	✔
CrossQ [1]	✔	✘	✘	✘	✔
DDPG	✔	✘	✘	✘	✔
DQN	✘	✔	✘	✘	✔
HER	✔	✔	✘	✘	✔
PPO	✔	✔	✔	✔	✔
QR-DQN [1]	✘	✔	✘	✘	✔
RecurrentPPO [1]	✔	✔	✔	✔	✔
SAC	✔	✘	✘	✘	✔
TD3	✔	✘	✘	✘	✔
TQC [1]	✔	✘	✘	✘	✔
TRPO [1]	✔	✔	✔	✔	✔
Maskable PPO [1]	✘	✔	✔	✔	✔

Choosing the right algorithm

Available algos in Stoix

Implementations of Algorithms 🥑

- **Deep Q-Network (DQN)** - Paper
- **Double DQN (DDQN)** - Paper
- **Dueling DQN** - Paper
- **Categorical DQN (C51)** - Paper
- **Munchausen DQN (M-DQN)** Paper
- **Quantile Regression DQN (QR-DQN)** - Paper
- **DQN with Regularized Q-learning (DQN-Reg)** Paper
- **Rainbow** - Paper
- **Recurrent Experience Replay in Distributed Reinforcement Learning (R2D2)** - Paper
- **REINFORCE With Baseline** - Paper
- **Deep Deterministic Policy Gradient (DDPG)** - Paper
- **Twin Delayed DDPG (TD3)** - Paper
- **Distributed Distributional DDPG (D4PG)** - Paper
- **Soft Actor-Critic (SAC)** - Paper
- **Proximal Policy Optimization (PPO)** - Paper
- **Discovered Policy Optimization (DPO)** Paper
- **Maximum a Posteriori Policy Optimisation (MPO)** - Paper
- **On-Policy Maximum a Posteriori Policy Optimisation (V-MPO)** - Paper
- **Advantage-Weighted Regression (AWR)** - Paper
- **AlphaZero** - Paper
- **MuZero** - Paper
- **Sampled Alpha/Mu-Zero** - Paper
- **Sequential Monte Carlo Policy Optimisation (SPO)** - Paper

Hyperparameter tuning

RL algorithms are sensitive to hyperparams

We need to find the best ones to reach the highest performance

Optuna – Library to automate the hyperparameter tuning using i.e Bayesian Optimization.

Reproducibility

The ability to obtain the same or similar results when running an algorithm on the same environment and under the same conditions

In computer science we get "pseudo-random numbers"

Examples:

- torch.manual_seed(seed)
- np.random.seed(seed)
- random.random.seed(seed)

Reproducibility

These pseudo-random numbers affect:

- Parameter initialization of the neural networks

- Initial conditions

- Target conditions

To make sure that our hyperparams are the best (and not only luck), we need to set the seeds to assure reproducibility

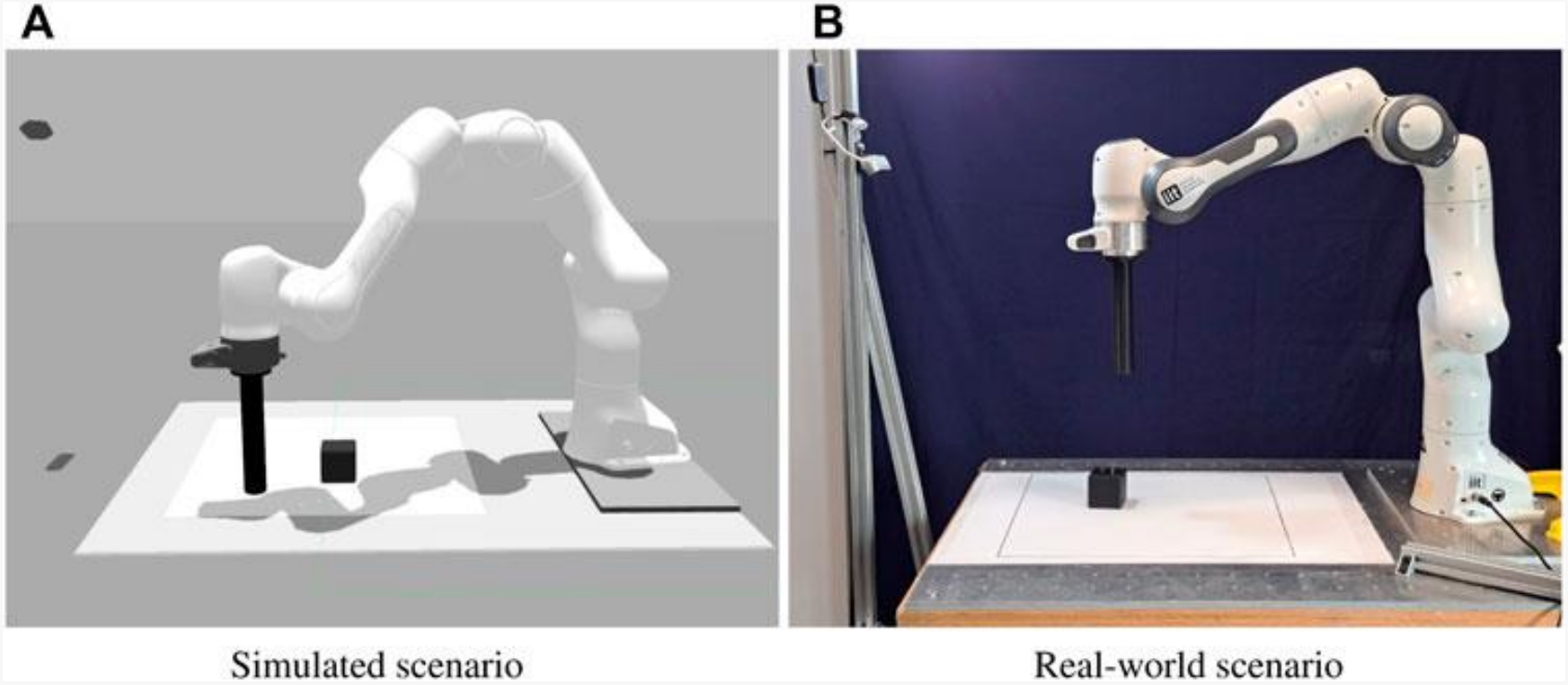
Simulation-to-Real

In RL we typically need **hundreds of thousands to millions of steps** to achieve good performance.

For that reason, we need to train in simulation

After training, we get the trained policy and apply it to the real world problem

Simulation-to-Real



Even though our simulators are so realistic, they typically cannot exactly match the reality

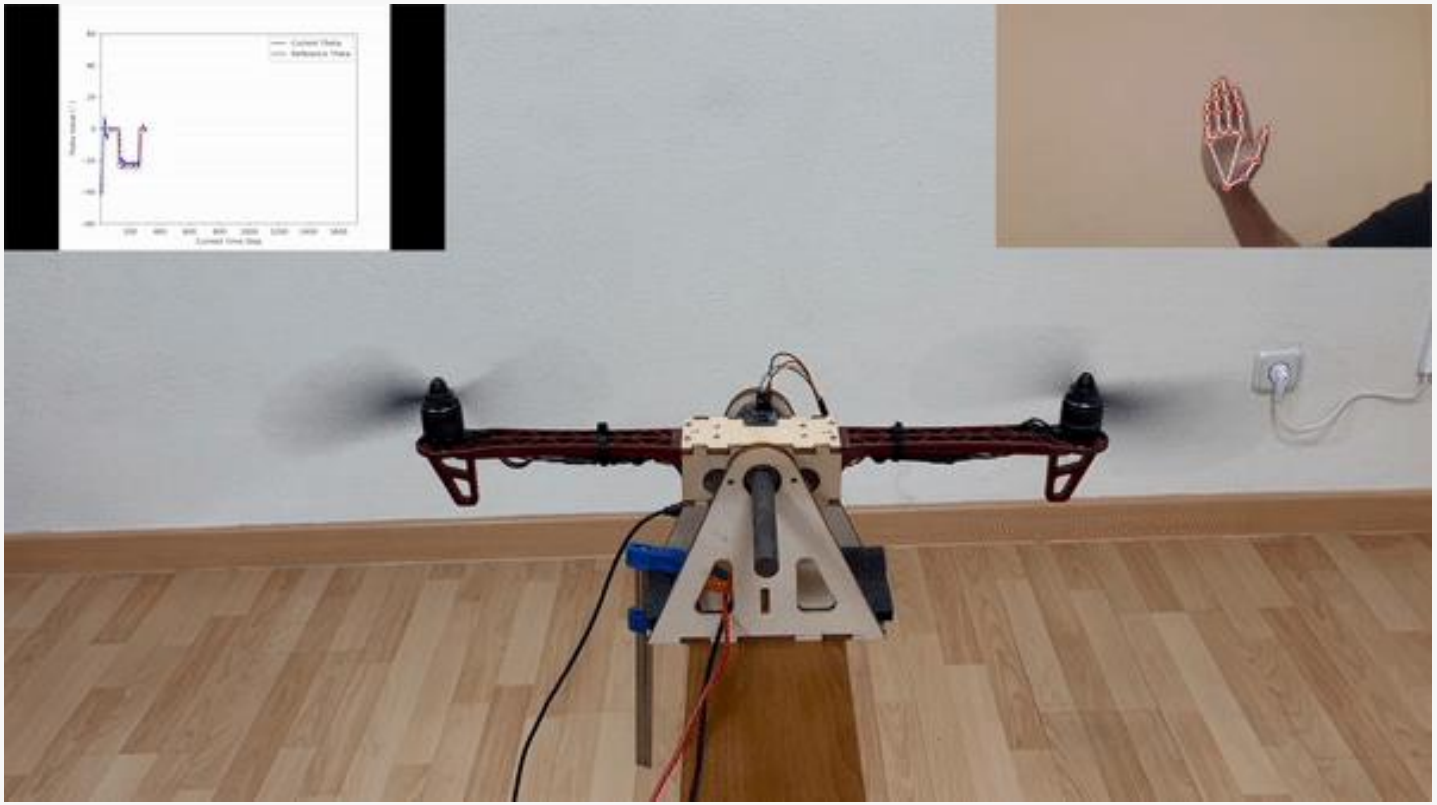
Some examples of implementations

Autonomous racing car

https://github.com/fgiral000/donkey_car_rl

Some examples of implementations

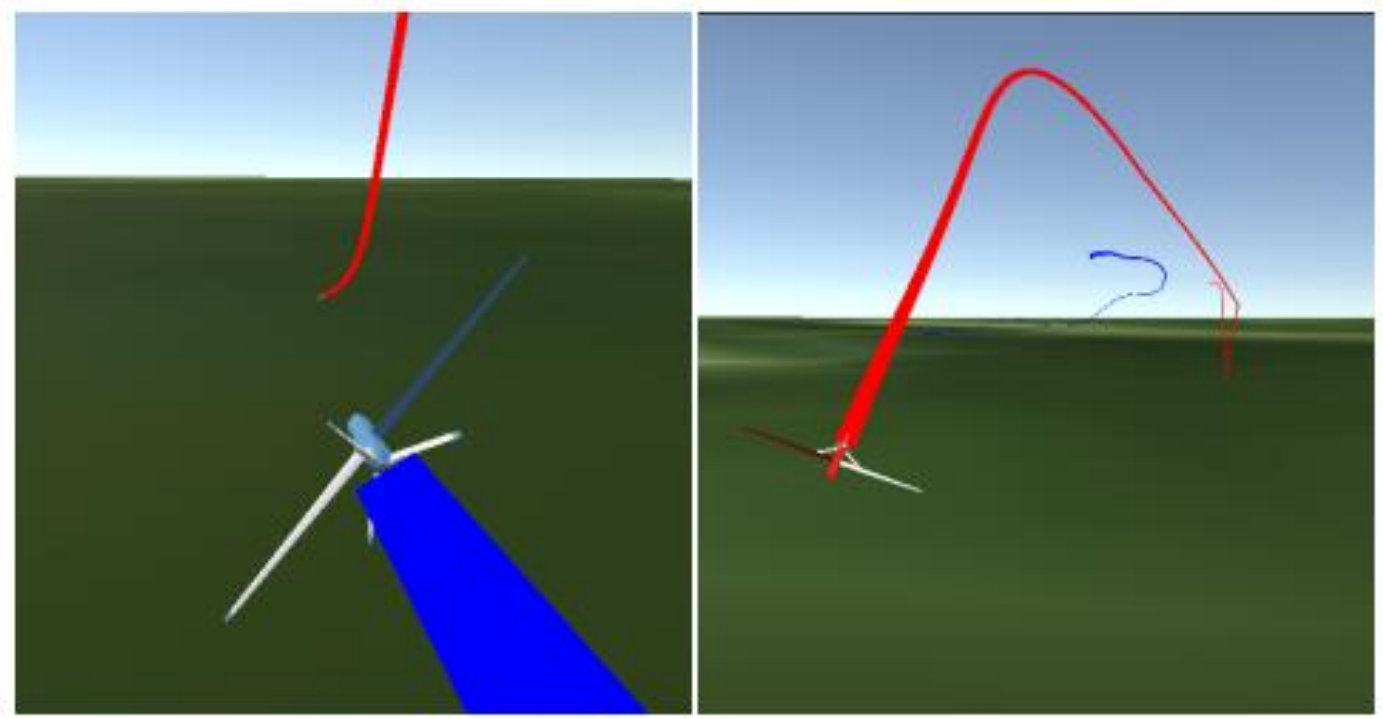
Training directly on real hardware (no simulation)



https://github.com/fgiral000/bicopter_RL_control

Some examples from our research

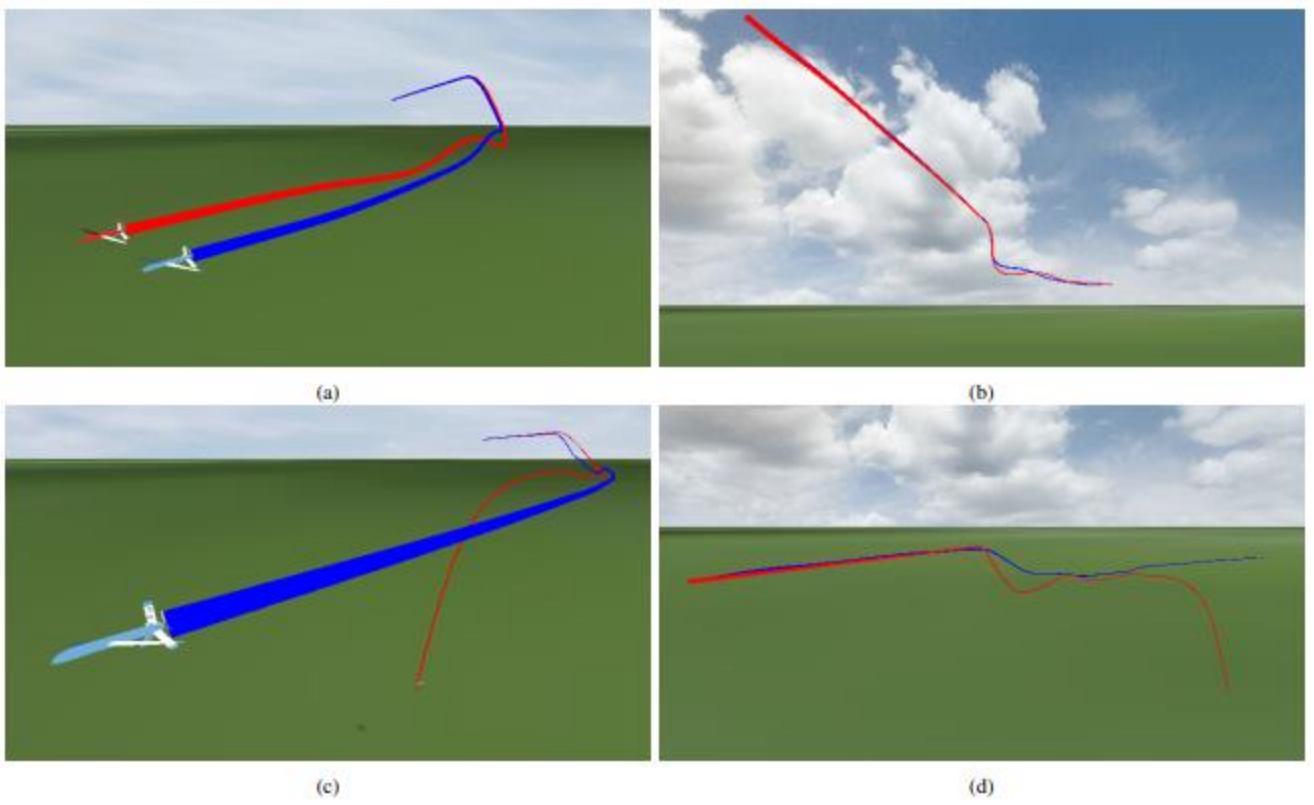
Intercepting a flying target



https://www.youtube.com/watch?v=dJd7lmK3QuU

Some examples from our research

Fault-Tolerant Flight Control



https://www.youtube.com/watch?v=ATW3LZFRqc0

Model-Based Reinforcement Learning

Imitation Learning / Offline RL

Multi-Agent RL

Acknowlegments

- CS 285 at UC Berkeley, Deep Reinforcement Learning. Sergey Levine
- https://araffin.github.io/slides/tips-reliable-rl/#/2/0/0
- Reinforcement Learning: A Comprehensive Overview. Kevin P. Murphy